

Insertions

The equal to (=) sign can be inserted, and the function type can be neatly aligned with the *C-c C-=* key stroke. If you type the following function:

```
volume :: Int -> Int -> Int
volume x
```

...and keep the cursor after 'x' and type *C-c C=*, the equal to sign is inserted, and the code gets neatly aligned:

```
volume :: Int -> Int -> Int
volume x =
```

In the following code snippet, after 'y', if you hit *Return* followed by *C-c C-/*, a guard symbol is inserted:

```
max :: (Ord a) => a -> a -> a
max x y
|
```

After inserting the second guard in the above example, the 'otherwise' keyword can be inserted and the code is aligned using *C-c C-o*:

```
max :: (Ord a) => a -> a -> a
max x y
| x > y     = x
| otherwise =
```

The 'where' clause is produced using *C-c C-w*. In the following example, pressing return after 'r', and using *C-c C-w* inserts the 'where' clause:

```
circleArea :: Float -> Float
circleArea r = pi * r * r
where
```

You can insert the type annotation for a function using *C-u C-c C-t*. Consider the *sphereVolume* function:

```
sphereVolume r = 4 / 3 * pi * r * r * r
where pi = 3.1412
```

Placing the cursor on 'sphereVolume' and typing *C-u C-c C-t* produces the following:

```
sphereVolume :: Fractional a => a -> a
sphereVolume r = 4 / 3 * pi * r * r * r
where pi = 3.1412
```

Formatting

There are a number of shortcut commands that are useful for indentation. Let's suppose you have the following function

with the cursor position indicated by '':

```
greeting :: String -> String
greeting x = "Hello" ++ x ++

```

Hitting *Tab* will take you through the different possible positions for inserting code. When you press *Tab* for the first time, the cursor will move under 'Hello'; if you wish to complete the string concatenation (++), issue the following code:

```
greeting :: String -> String
greeting x = "Hello" ++ x ++

```

Hitting *Tab* again prepends 'greeting' and the cursor will be placed under 'x' for you to add another test condition, as follows:

```
greeting :: String -> String
greeting x = "Hello" ++ x ++
greeting _
```

Hitting *Tab* yet again will move the cursor to the first column if you want to add any text:

```
greeting :: String -> String
greeting x = "Hello" ++ x ++

```

As you keep hitting *Tab* again and again, the above sequence will repeat. Comments in Haskell begin with '*--*'.

```
one -- 1
two  -- 2
three -- 3
four  -- 4
five  -- 5
six   -- 6
seven -- 7
```

After marking the above region, use *M-x align-regexp* followed by '*--*' for the regexp, and the comments will be aligned:

```
one    -- 1
two    -- 2
three  -- 3
four   -- 4
five   -- 5
six    -- 6
seven  -- 7
```

C-c C- helps align the code neatly. Consider the *Area*. *hs* program:

```
area :: Int -> Int -> Int
area breadth height = breadth * height
```